



Toc H Institute of Science & Technology
(An ISO 9001:2015 Certified Institution)
Arakkunnam

Date: 25/8/25



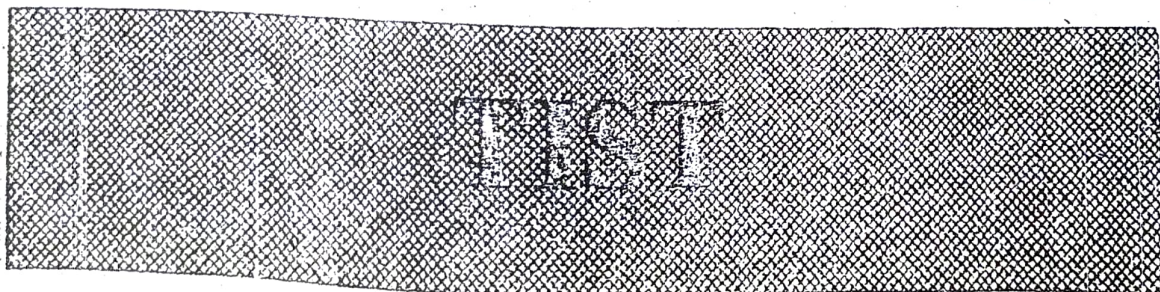
INTERNAL ASSESSMENT MAIN ANSWER BOOKLET

	I	II	III	IV	V	VI	VII	VIII	IX	X	XI	XII	XIII
a	3	3	1	3	72	6	3	1					
b						7	8						
c													
d													
e													
f													
g													
h													
Total													
Grand Total													

TOTAL

35/2
56

352



Name	MADHAV J NAIR												
Branch & Semester	CSE 55 B												
Subject with Code	CST 305 SYSTEM SOFTWARE												
Roll No.	18												
Signature of Invigilator													
University Reg. No: T O C 2 3 C 5 0 8 2													
No. of Additional Sheets													

a)

There are ~~4~~ ^{types} multiple types of ~~system~~ ^{software} ~~software~~ [&] ~~it~~

System software : A software that provides an interface between Application software & hardware.

The examples of system software are

OS : An operating system is a software that provides a base for application software to run on a hardware. It performs multiple functions such as, It is built on top of a kernel
Device management, process management, file management, memory management, authentication, etc.

Device Driver : These are system software that ~~are used~~ ^{various} helps ~~the~~ ^{various} softwares to interact with the input devices. Many of the device drivers are preinstalled with OS, however some device drivers are to be installed by the user.

Assembler : The job of an assembler is to convert Assembly language to object program.

There are single pass assembler, multi-pass assembler or two pass assembler based on number of passes.

There are multiple types of assembler such as ~~si~~ self assembler, cross assembler, etc.

Compiler : A compiler converts a High Level Language code to a ~~low~~ ~~level~~ machine language.

It converts the whole file into an executable (.exe) file initially Δ then to machine code.

6 b) Memory :

The SIC/XE machine has 2^{20} bytes i.e. 1 MB of storage.

Registers :

The SIC/XE machine has following registers

A → Accumulator
 X → Index
 L → Linkage (used for Jump)
 PC → Program Counter (stores address
 of next instr.)
 SW → Status word
 B → Base
 S, T → general purpose
 F → Floating point

} only for
 SIC/XE

~~Address~~

Address format:

~~Addresses can be stored in 8 bit~~
 words are stored in 8 bit format
 integers are stored in 24 bits
 with 1 bit extra that shows
 the or the sign

floating point numbers are stored
 in 48 bits → only for SIC/XE

~~Addressing mode~~

Instruction Set:

Load and store instruction: used
 to store & load data from
 registers & back

eg: LDA, STA, LDX, STX, etc

~~Register~~ to Register instructions:
 used to transfer data from
 one register to another (only
 for SIC/XE)

Arithmetic Instruction : Used to perform arithmetic operations

eg: ADD ~~R~~, SUB, MUL, DIV, etc

Register to Register Arithmetic : Used to perform arithmetic operations b/n ~~the~~ contents of two registers
Only for SIC/XE)

eg: ADDR, SUBR, etc

Compare instructions : To compare the data b/n 2 registers

eg CMP

Jump Instruction : ~~Used~~ Used ~~in jumps~~ to jump from one statement to a statement which is not the next statement, ie it skips statements in b/n

eg: JMPZ, etc

Linkage based instruction : Used to jump while using Linkage register

eg JSUB, etc

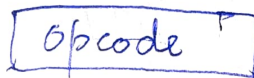
TD : This instruction is used to test the input device to check if it's ready

MADHAI J.N.M.K

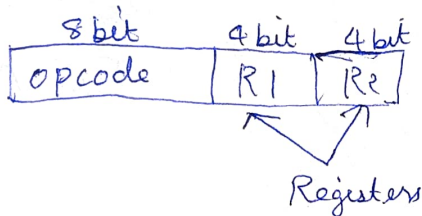
I/O instruction : Used to accept ¹⁸ CSE S&B
input from various device
give out output.

Instruction formats

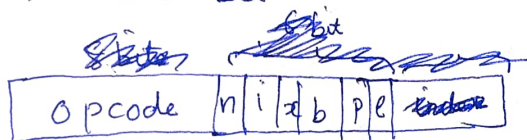
Format 1 : 8 bit



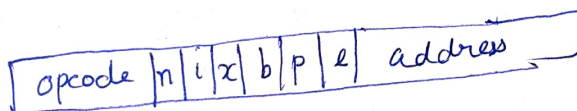
Format 2 : 16 bit



Format 3 : 24 bit



Format 4 : 48 bit



~~Addressing~~ Addressing modes :

- 6

1. PC relative
 2. Base relative
 3. Direct
 4. Indirect
 5. Indexed
 6. Base relative indexed
 - 7.

& I/O : SIC / XE can take input from multiple input devices

7a)

~~ALPHA~~ LDA ~~ALPHA~~ BETA

MUL #4

SUB #9

STA ALPHA

~~ALPHA~~

ALPHA ~~RESW~~ RESW

~~ALPHA~~ ~~RESW~~

BETA RESW

7b)

~~ALPHA~~

start
begin

read first line

if { ~~OPCODE~~ ^{OPERAND} = START }

initialise ^{#OPERAND} starting address

write the starting address

~~write~~ read next line to LOCCTR

~~end~~ end { if OPCODE = START }

else

initialise LOCCTR to 0

while { OPCODE != END }

begin

if not a comment line

begin

if it is a symbol in
label field

begin

search for symbol in
~~OPCTAB~~ SYMTAB

if found

write ~~symbol~~ ~~to~~ ~~LOCCTR~~

else

MADHAV J NAO
CSE 558
18

add { LABEL, LOCCTR }

to SYMTAB

end { if ~~next line~~ symbols
search OPERAND in OPTAB

~~if~~ if OPERAND = ' ~~RESB~~ ~~RESW~~ ' ~~WORD~~

Increment LOCCTR by 3

else if OPERAND = 'RESB'

Increment LOCCTR by
~~#[OPERAND]~~ the
no of bytes

else if OPERAND = 'RESW'

Increment LOCCTR by
 $3 * \#[\text{OPERAND}]$

else if OPERAND = 'CHAR'

find out the ~~number~~
space taken by char
write it to LOCCTR

else

write error flag

~~read~~ read next line

end { if not comment line }

read next line

end { if comment line }

end while loop

store length as LOCCTR - starting address

end

DATA STRUCTURES

OPTAB - It stores details of the operands, predefined
SYMTAB - stores the symbol & its location, used to help
LOCCTR - stores the forward reference to the address

1. System software

- It provides ~~platform~~ ~~for~~ interface b/w application software & hardware
- written in low level code due to its interaction with hardware
- It can work independently & it provides platform for application software

eg: Linker, loader, OS, etc

Application software

It provides a solution to a particular problem of the user

written in high level code ~~data~~

It cannot work independently & depends on system software

eg: Word, PPT, Browser, etc

2. START - It is used to mention the start of a program

END - It mentions the end of a program

RESB - It is to reserve bytes of specified lengths

3) The ~~form~~ format is Header, Text,

End

#

~~Header~~

Format of

Header:

H¹ <name of program>¹ <starting address>¹
<length of program>

T¹ <starting address of Text record>¹

<object codes ^{upto} 60 columns>

E¹ <starting address>

4)

LDA ALPHA

STA GAMMA

LDA BETA

STA ALPHA

LDA GAMMA

STA ALPHA

RESW ALPHA

RESW BETA

RESW GAMMA

8

Location

Label

Opcode

Operand

Operand

4000

TEST

START

4000

4000

FIRST

LDA

FIVE

004000

4002

STA

ALPHA

4008

ALPHA

RESW

2

400B

FIVE

WORD

5

5

RMO S, A

format: 2

~~AC~~

opcode	R1	R2
--------	----	----

AC

-JSUB RDREC

format 4

opcode	n	i	x	b	p	e	address
--------	---	---	---	---	---	---	---------

~~AC~~

1100011 01036

1036

~~AC~~ is b/a

-2048 & 2048

~~AC~~

LDA #1

Format 3

opcode	n	i	x	b	p	e	7
--------	---	---	---	---	---	---	---

101000